

1 (50 marks) Heaps, Hashing and A* search

In this problem you're going to be putting a lot of your knowledge together and make use of heaps, hash tables and the A* algorithm to solve the [8 puzzle](#).

1.1 8 Puzzle Problem Description

The 8-puzzle is a sliding puzzle that is played on a 3-by-3 grid with 8 square tiles labeled 1 through 8, plus a blank square. The goal is to rearrange the tiles so that they are ordered from 1-8 across rows, using as few moves as possible. You can slide tiles either horizontally or vertically into the blank square. The diagram in figure 1 shows a sequence of moves from an initial board (left) to the goal board (right).

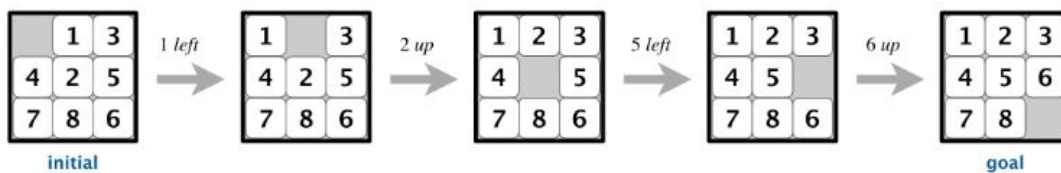


Figure 1: 8 Puzzle Problem Example

Note that we can extend this to the 15 puzzle or larger puzzle sizes.

1.2 Solving the 8 Puzzle Problem

Similar to what we've already seen in the chess example, we can consider this a graph search problem, where nodes in the graph represent different game states. Note that this graph will *not* be a tree since we can create a cycle by 'undoing' a move.

In order to solve the problem, you will need to search through the graph from the initial state to the goal state. Ideally we want to reach the goal state in as few moves as possible. A well-implemented A* search algorithm can do this for us. We're going to write our own version of A* with our own heap to implement the priority queue. We're then going to use a hash table to store the board states we've already visited so that we don't visit them again. The details for how you're going to implement each are below:

1.3 Representing the Board State

The board state can be represented as a nxn matrix (for the 8 puzzle this will be 3x3 but we can also solve the 15 puzzle using 4x4). You can represent this using a 2D array (each of size 3) or a 1D array of size 9. The 2D approach has been provided. This matrix will be used as the underlying data structure in the wrapper class provided. You will need to implement all of the methods with missing code. You may implement as many private helper methods as you want. Some notes:

- We'll represent the blank square as a 0.
- You can move the blank square up, down, left or right (but not beyond the edges obviously). The swap method provided will do this for you.
- Since we'll be checking if we've already visited a state, you'll need to implement a hash function for the board state. You know that this means the board state should be immutable relative to how you're computing the hash.
- We'll need to check if the board state is the goal state, which will require you to override the `.equals()` method. This has been partially implemented for you.

1.4 Implementing the A* Algorithm

To run A* search, you know that you'll need to implement a search heuristic which underestimates the distance to the goal state. There are three methods below that you will need to implement:

1. The manhattan distance is the sum of the absolute differences between the x and y coordinates of the tiles in the current state and the goal state.
2. The hamming distance is the number of tiles that are out of place in the current state compared to the goal state.
3. The sum of the manhattan and hamming distances.



Figure 2: Board State Example

In the example shown above in 2, the manhattan distance to the goal is 10 (tile 1 is 1 away, tile 2 is 2 away, tile 5 is 2 moves away, tile 6 is 2 moves away, tile 8 is 3 moves away). The hamming distance is 5 since 5 tiles are out of place. The sum of the manhattan and hamming distances is 15.

Out of these three methods, give both empirical and theoretical reasons for which one you would choose to use as your heuristic using the following criteria:

1. Are they consistent (i.e. is its estimate is always less than or equal to the estimated distance from any neighbouring vertex to the goal, plus the cost of reaching that neighbour?)
2. Will one be faster to compute than the other?
3. Will one expand fewer nodes than the other?

In the skeleton code provided, you can specify which of these you'll be using. You need to complete the skeleton code so that the tests pass. To implement A* you'll need to use a priority queue. This has been provided, **do not use any built-in or 3rd party priority queues**. You'll also need a hash table to record the states you've already visited so that they don't get endlessly revisited. You'll need to implement a hash function for the board state to use this hash table. Two implementations have been provided (separate chaining and quadratic probing.) You need to choose one and provide justification behind your choice.

1.5 Hash Table Implementation

You've been given both a quadratic probing and separate chaining hash table implementation. You'll need to decide on both the size of the hash table you're going to use as well as a hash function. Justification will be needed for both and marks will be awarded for efficiency. You are also welcome to try to implement perfect hashing, just make sure you explain what you have done if you do attempt it.

1.6 Priority Queue Implementation

You've been provided with a binary heap implementation. You'll need to decide on the size of the heap you're going to use. You must use this binary heap in your A* algorithm, **you may not use any built-in classes or data structures**.

1.7 Grading

This section will be graded according to the following:

1. (10 marks) Test suite passes (you may not modify the test suite unless you clear it with me first)
2. (5 marks) All methods with code missing implemented
3. (10 marks) Correct implementation of the A* algorithm (i.e. shortest path to solution found)
4. (10 marks) Correct implementation of the hash table with good choice of hashing function (limited collisions & efficient to compute. Keep the load factor/clustering small if possible).
5. (10 marks) Theoretical and empirical analysis of heuristics.
6. (5 marks) Originality and style of code (i.e. good use of helper functions, readability, descriptive comments etc.).

1.8 Bonus Problem - 2 quiz points

Extend the code to implement an adversary which randomly moves a square between your turns. Come up with an algorithm which can still solve the puzzle.

1.9 Second Bonus Problem - 2 quiz points

Use a suitable data structure to record the order of the moves. Once the algorithm has run you should be able to call a methods which says ‘move tile 1 to the empty tile, move tile 4 to the empty tile, etc.’

2 (50 marks) Sorting

2.1 (30 marks) Custom Sorting Algorithms

We’ve studied sorting algorithms in the lecture. We’re going to experiment with 2 different sorting methods:

1. Hybrid Merge/Insertion Sort
2. Hybrid Quicksort/Insertion Sort

Please note that everything will be done in-place.

2.1.1 (15 marks) Hybrid Merge/Insertion Sort

In this question you need to implement a hybrid sorting procedure, which takes advantage of the strengths of both merge sort and insertion sort. You should implement the sorting procedure as follows:

1. Recursively split the list into two halves until the length of the sub-list is less than 10.
2. Recombine the sub-lists as follows:
 - If the length of the list is less than 10, use insertion sort to sort the sub-list. Your insertion sort algorithm should use binary search to find the correct index to insert the element.
 - Once adjacent smaller sub-lists of size < 10 have been sorted, merge them together using the merge sort algorithm.

You will need to complete all the methods with missing code in the code skeleton provided. You’ll need to keep track of the number of times the merge and insertion sort procedures are called. Some tests cases have been written which will need to pass for you to get full marks. If you believe there is a mistake in the test cases contact me.

2.1.2 (15 marks) Hybrid Quicksort/Insertion Sort

In this question you need to implement a hybrid sorting procedure, which takes advantage of the strengths of both quicksort and insertion sort. You should implement the sorting procedure as follows:

1. Use quicksort to fix a pivot element and partition the array into two sub-arrays.
2. If any sub-array is smaller than size 20, use insertion sort to sort the sub-array. Your insertion sort algorithm should use binary search to find the correct index to insert the element.

You will need to complete all the methods with missing code in the code skeleton provided. You'll need to keep track of the number of times the partition via a pivot and insertion sort procedures are called. Some tests cases have been written which will need to pass for you to get full marks (if yours are failing due to randomization you can note why).

2.1.3 Grading

This section will be graded according to the following:

1. (10 marks) Test suite passes (you may not modify the test suite unless you clear it with me first)
2. (5 marks) All methods with missing code implemented
3. (15 marks) Originality and style of code (i.e. good use of helper functions, readability, descriptive comments etc.).

2.2 (20 marks) Efficient Colinear Point Detection Using Sorting

Computer vision algorithms often require some kind of pattern recognition as a preliminary phase of the algorithm. Statistical analysis techniques also often look for patterns. One such pattern is finding a sequence of points which are *collinear* i.e. they all lie on the same line segment. In this problem you'll be using one of the sorting algorithms you implemented above to find collinear points in a set of points.

2.2.1 Problem Description

The problem you need to solve is as follows:

Given a set of n distinct points in the 2-D plane, find every (maximal) line segment that connects a subset of 4 or more of the points. Here maximal means that the line segment cannot be extended by adding another point to the subset.

2.2.2 Point and Line Segment Representation

Important: You must use java 21 to run this code!

You have been provided with code for the point and line segment representation. You will need to use these classes to find colinear points.

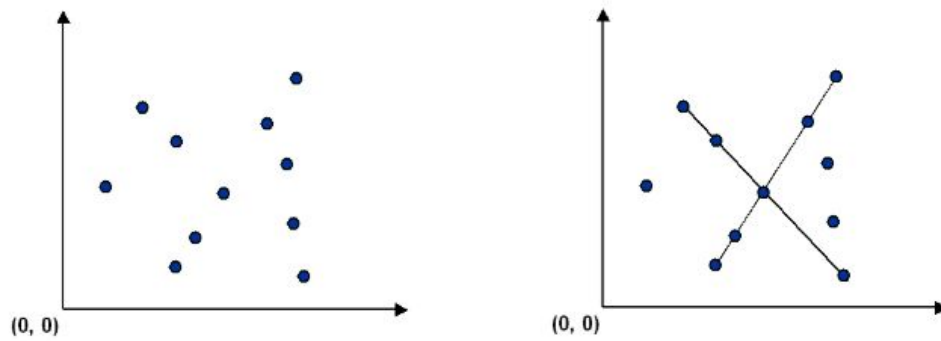


Figure 3: Example of collinear points

2.2.3 Brute Force Solution

First, you should implement a brute force algorithm by checking every possible combination of 4 points to see if they are collinear. To do this, look at 4 points at a time and record whether they all lie on the same line segment.

If 4 points appear on a line segment in the order $p \rightarrow q \rightarrow r \rightarrow s$, then you should include either the line segment $p \rightarrow s$ or $s \rightarrow p$ (but not both) and you should not include subsegments such as $p \rightarrow r$ or $q \rightarrow r$.

2.2.4 Efficient Solution Using a Sorting-Based Algorithm

In the second part, you need to implement an efficient solution in the *FastCollinear* class. You need to use the sorting algorithm from the previous part. **Do not use the built-in sorting methods.** The brute force method is inefficient because it takes $O(n^4)$ time to check all possible combinations of 4 points (we could speed up a little by not doing some repeat checks but it's still bad!). Given a point p , the following method determines whether p participates in a set of 4 or more collinear points:

1. Think of p as the origin.
2. For each other point q , determine the slope it makes with p .
3. Sort the points according to the slopes they makes with p .
4. Use a 'sliding window' to check if any 3 (or more) adjacent points in the sorted order have equal slopes with respect to p . If so, these points, together with p , are collinear.

Applying this method for each of the n points in turn will yield an efficient algorithm to the problem. See figure 4 to understand how it works. The algorithm solves the problem because points that have equal slopes with respect to p are collinear, and sorting brings such points together. The algorithm is fast because sorting removes the need for searching through everything.

Performance requirement. The order of growth of the running time of your program should be $n^2 \cdot \log(n)$ in the worst case and it should use space proportional to n plus the number of line segments returned. *FastCollinear* should work properly even if the input has 5 or more collinear points. Note:

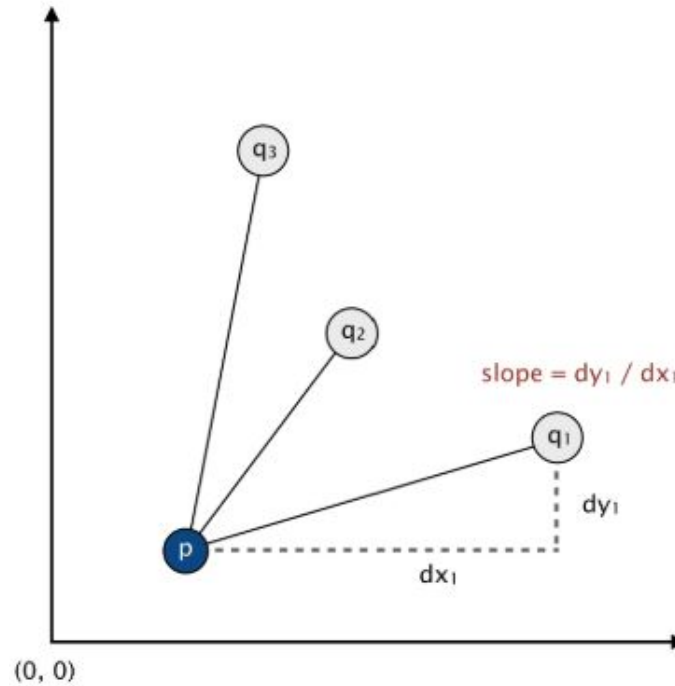


Figure 4: Efficient collinear points detection example

you may assume the sorting methods that you implemented run in $O(n \cdot \log(n))$ time (even if you realize this isn't true).

2.2.5 Grading

This section will be graded according to the following:

1. (5 marks) You pass my test suite (not provided! But very important you don't change any public method signatures so I can run the tests)
2. (10 marks) All methods with missing code implemented.
3. (5 marks) Performance meets the required standard. Originality and style of code (i.e. good use of helper functions, readability, descriptive comments etc.).

2.3 Submission

You need to submit **1 pdf and 1 zip file**:

1. The zip file should contain all of your code as a maven project (including tests, test data, etc). I need to be able to unzip and run it without any modification. It should all be contained in the same maven project I have provided you. If I cannot run your code without modification you will lose marks for the 'test suite passes' sections. If you're unsure about this, please ask me.

2. The pdf should contain the relevant bits of code you have written for each section. Please make it clear which sections your code is for. Any extra analysis/justification should also be included in the pdf.

If you realize there are any problems with the code or tests please ask me about it, I have not tested everything super thoroughly!